ChatGPT

APA League Data JSON Structure Analysis

Overview of the JSON File Structure

The `api_dump.json` is a **JSON array** where each element captures a GraphQL API call made by the APA league web app. Each element includes:

- `page_url` – the web page URL that triggered the API call (e.g. a division standings page or match page).
- `api_url` – the endpoint for the GraphQL API (all calls use the same `/graphql` endpoint).
- `type` / `method` / `status` – HTTP request info (all are `fetch` POST requests with status 200 in this file).
- `request_body` – an array of one or more GraphQL query objects sent in the request. Each object has an `operationName` and the `query` (with variables).
- `response_body` – an array of corresponding responses, typically containing a top-level `data` object with the requested fields.
- `headers` – HTTP response headers (content type JSON, etc.).

Hierarchy: The meaningful APA data resides inside the `response_body`'s `data` fields. We will focus on the data returned, which is structured around APA league concepts (leagues, divisions, teams, players, standings, matches, stats). The key sections of the JSON data hierarchy are:

- **Viewer/Member and League info** – identifies the logged-in member and available leagues.
- **Divisions** – lists of divisions in a league, and details for a specific division (format, schedule, contacts).
- **Teams and Standings** – teams within a division and their standings (ranks, points).
- **Team Rosters** – players on a team roster with stats.
- **Match Details** – information about specific team vs. team matches (including scores and player performance).
- **Player (Member/Alias) Stats** – individual player statistics and history (lifetime stats, session stats, membership info).

We'll break down each section, list the relevant JSON fields, their data types, purpose, and how they relate to APA concepts. We'll also discuss how these might map to data models or endpoints in a Flask app.

Viewer and League Metadata

At the top of the JSON, the app first retrieves info about the current user (viewer) and their leagues. In the `response_body` for an initial request, we see:

- `viewer.id` (Integer) – The member ID of the logged-in user (e.g. `id: 3400522`) [1]. This is a global APA member identifier.
- `viewer.__typename` – The type of user; in our case `"Member"` (as opposed to staff types) [1].

- `viewer.leagues` – An array of League objects that the member belongs to. In this file, there is one league:
- `leagues[].id` (Integer) – League ID (e.g. `894`) [2].
- `leagues[].slug` (String) – URL slug for the league (e.g. `"RioGrandeValley"`) [2].
- `leagues[].name` (String) – Full name of the league (e.g. `"Rio Grande Valley APA"`) [3].
- `leagues[].isDefault` (Boolean) – True if this is the user's default/home league [3].
- `leagues[].isElectronicPaymentsEnabled` (Boolean) – Indicates if the league accepts electronic payments [2].

These fields provide context for the rest of the data. In a Flask app, one might use this to determine which league's data to show by default. The **League ID (`894`)** will be used as a key for fetching divisions, teams, etc., and the **member ID** could link to player data.

Divisions List and Division Details

Divisions represent competitive groupings within the league (often based on night of play, format, location, etc.). The JSON includes a "divisions dropdown" query which returns all divisions for the current session. Each division object includes:

- `division.id` (Integer) – Unique division identifier (e.g. `417524`) [4].
- `division.name` (String) – Division name (e.g. `"Thursday Brownsville 9 B"`) [5].
- `division.number` (String) – Division number/code as assigned by APA (e.g. `"222"`, sometimes combined with team numbers) [5].
- `division.format` (String) – Game format, e.g. `"8-Ball Open"` or `"9-Ball Open"` [6]. This indicates whether the division plays 8-ball or 9-ball (Open denotes standard format).
- `division.nightOfPlay` (String) – Night of week for play (e.g. "Thursday").
- `division.timeOfPlay` (String) – Typical start time.
- `division.isMine` (Boolean) – True if this division is "mine" (the user's team plays in it) [7].
- `division.state` (String) – Current state of the division's session (e.g. active, finished, etc.).
- `division.schedule` – Object with schedule info (e.g. `schedule.id` for the schedule, if needed).
- `division.league` – Nested League reference (with `id` and `slug`) linking back to the parent league [8].

For example, one division entry in the divisions list appears as:

> **id:** 417524, **name:** "Thursday Brownsville 9 B", **number:** "222", **format:** "9-Ball Open" [5].

Another: **id:** 417530, **name:** "Thursday Brownsville DJ", **number:** "022", **format:** "8-Ball Open" [9] [6]. (The "DJ" indicates a Double Jeopardy division, which has both an 8-ball and 9-ball side; note the division numbers `"222"` vs `"022"` for the 9-ball vs 8-ball side).

Division Contacts: Each division can have contacts (like a Division Representative or League Operator). The `DivisionContacts` query returns an array of contacts for a division:

- `division.contacts` – List of contact entries, each with:
- `phone` (String) – Contact phone number.

- `alias` – An Alias object for the contact person, containing `id` and `displayName` (person's name) [10] [11].

In our data, most division contacts arrays are empty (e.g. `"contacts": []` for division 417530) [11], meaning no specific contacts listed for those divisions. A Flask app might display contact info if available (for example, "Division Rep: [Name] – [Phone]").

Usage in Web App: The list of divisions (id, name) is used to populate dropdowns or navigation so the user can select a division. When a specific division page is loaded, additional queries fetch details like standings and schedule for that division. The division's `format`, `nightOfPlay`, etc., help label the page (e.g. "8-Ball Open – Thursdays"). Division `id` is a key for further queries (standings, schedule, etc.) and for linking teams to their division.

Teams and Standings in a Division

Within a division page (e.g. the Standings page), the JSON provides a **standings list** of all teams in that division and their current stats. This comes from the `divsionStandings` (sic) query. Each **Team** object in the standings includes:

- `team.id` (Integer) – Unique team identifier (e.g. `12864635`) [12]. This ID is used in match data and could be used for team-specific endpoints.
- `team.name` (String) – Team name (e.g. `"Chalk and Awe"`, `"Cue-ligans"`) [13].
- `team.number` (String) – Team number within the division, typically a concatenation of division number and a team sequence (e.g. `"22202"` for division 222 team #02) [14].
- `team.standing` (Integer) – Current rank/place in the division (1 = first place) [15].
- `team.pointsLastWeek` (Integer) – Points earned in the most recent week of play [16]. For example, Chalk and Awe had `"pointsLastWeek": 65` [13].
- `team.lastWeek` (Integer) – The week number of the most recent match played. In our data all teams show `lastWeek: 11` (meaning the standings are as of Week 11) [17].
- `team.sessionTotalPoints` (Integer) – Cumulative points the team has earned in the session (season) [18].
- `team.totalTeamMatchesPlayed` (Integer) – Number of team matches played so far [19] [20]. This correlates with weeks played (e.g. 55 matches played suggests 11 weeks × 5 individual matches each week for 9-ball; values differ if bye weeks or forfeits occurred).
- `team.isTied` (Boolean) – True if the team is tied with another in points/standing [21].
- `team.isBye` (Boolean) – True if this entry is a placeholder "Bye" team (for divisions with an odd number of teams) [22]. In our data all actual teams have `"isBye": false`.
- `team.league` – Nested league reference (id and slug) linking back to the parent league [21].
- `team.tournaments` – An array (often empty) for any future tournaments the team is involved in [19]. This could list higher-level tournaments (e.g. City Cup, etc.) if applicable. In our sample, `"tournaments": []` for all teams in standings.

These fields align with APA's weekly standings: for example, a team named **"Chalk and Awe"** might be 1st place with 758 points total and 65 points gained last week [13]. Another team **"Cue-ligans"** might be 4th with 720 points [23], etc. The JSON snippet below illustrates two teams' standings:

[13] [24]

In a Flask app, this data would be used to render a standings table. Each team's ID can be used to link to a team detail page or schedule. The **team number** is important in APA as it appears on score sheets (combining division and team). The `isBye` flag indicates if a "Bye" occupies a slot in the schedule (which usually has no players and always 0 points).

Identifier Relationships: The team entries connect to other entities: each team knows its `division.id` (through context or the nested division in team queries) and `league.id`. The team `id` is used in match records and could be used as a foreign key in a database (Team table).

Team Roster and Player Statistics

For each team, especially when viewing a match or team page, the app retrieves the **roster** – the list of players on that team for the session, along with their current stats (often used for choosing players in a match lineup or viewing individual performance). In our JSON, roster data appears as part of the match details (embedded under each team in a match). A Team's roster is an array of **Player** objects. Key fields for each player in the roster:

- `player.__typename` – Indicates format-specific player type: `"EightBallPlayer"` or `"NineBallPlayer"`. This distinction is important because stats fields differ between 8-ball and 9-ball players.
- `player.displayName` (String) – The player's name (could be first + last or nickname) [25].
- `player.id` (Integer) – The player's unique ID for that session and format. For example, Justin Ross as an 8-ball player has `id: 91040482` [25]. These IDs are high numbers and are distinct from member or alias IDs.
- `player.matchesPlayed` (Integer) – Number of individual matches the player has played this session [26].
- `player.matchesWon` (Integer) – Number of individual matches won by the player this session [26].
- `player.skillLevel` (Integer) – The player's current skill level in this format [27]. (APA skill levels range 2–7 for 8-ball, 1–9 for 9-ball).
- `player.member` – A nested Member reference with at least an `id` (the APA member ID) [28]. This links the player to the person's global identity.
- `player.memberNumber` (String) – The APA membership number (often a composite of league and personal ID) [27]. In our data, memberNumber looks like "785xxxxx" which is an APA number including area code 785.
- `player.pa` (Number) – **Points Average** [27]. For 8-ball, "PA" is a metric APA uses (points earned divided by matches; in 8-ball, usually 3 points for a shutout win, etc.). It's given as a decimal (e.g. Justin Ross has `pa: 0.2917` which might correspond to roughly 2/7 points average) [27].
- `player.ppm` (Number) – **Points Per Match** [29]. Another performance metric (total points divided by matches). For instance, William McGinnis has `ppm: 1.2` in 8-ball [30], meaning he averages 1.2 points per match. These metrics help compare player contributions.

An example roster entry for the team *Cue-ligans* in an 8-ball division:

[31]

Here, “Justin Ross” has played 8 matches with 1 win, SL3, and PA 0.2917, PPM 0.875 [26]. Another player “William McGinnis” is SL5 with 5 matches, 3 wins, PA ~0.40, PPM 1.2 [32] [30]. These fields correspond to the stats one would see on a weekly score sheet or roster summary (APA often tracks total points and averages).

All players on the team are listed similarly. The roster information is typically used in the app’s **Score Input** interface – when entering a match, the roster is shown to select which players played. It can also populate a team roster page if one exists.

Relationship of IDs: Each player’s `member.id` ties to the main Member record (e.g. Justin Ross’s member id is `3400522` which matches the viewer’s id in this case) [33]. The `player.id` is a session-specific identifier. The `alias` concept appears here indirectly: each player on a roster corresponds to an alias (the person in context of this league). In fact, the **alias ID** is not explicitly in the roster entry; instead we have member ID and the player’s session record. For deeper stats, the app uses the alias ID (see next section).

From a Flask modeling perspective, one might have separate tables for **PlayerStats** per session or embed these stats in a join table between Alias and Session. Each roster entry essentially links: Team <- Session -> Alias (Member). A possible schema: `TeamMembership(alias_id, team_id, session_id, matches_played, matches_won, skill_level, PA, PPM, ...)`.

Match Details and Structure

A significant portion of the JSON is dedicated to **match details** – i.e. the results of a specific weekly match between two teams. Each match is identified by a `match.id` (e.g. `48118020`) and contains comprehensive information: teams involved, location, date, scores, etc. The `MatchPage` query returns a `match` object with the following key fields:

Basic Match Info:
- `match.id` (Integer) – Unique match identifier (e.g. `48118020`) [34]. This ID is used in URLs (`/match/48118020`) and links schedules and score sheets.
- `match.session` – Session object (with `id` and name/year) indicating which session (season) this match is in. For example, `"session": {"id": 137, "name": "Fall 2025"}` [35].
- `match.week` (Integer) – The week number of play (e.g. `week: 1` for the first week of the session) [35].
- `match.startTime` (DateTime String) – Scheduled start date/time for the match (ISO format) [35].
- `match.timeZone` – Time zone info (the example shows America/Chicago) [36].
- `match.type` (String) – Format of the match, e.g. `"EIGHT"` or `"NINE"` [36].
- `match.isFinalized` (Boolean) – True if the match scores have been finalized (submitted/approved) [37]. Once finalized, further edits are locked.
- `match.isScored` (Boolean) – True if scores have been input (the example shows `isScored: true` for a completed match) [38].
- `match.isPaid` (Boolean) – True if match fees have been paid (our example shows `isPaid: true` for the match, meaning the weekly dues were paid online) [38].
- `match.isTournament` (Boolean) – False for regular league matches (True if this was a tournament match). All matches here are league matches (`isTournament: false`) [38].
- `match.isBye` (Boolean) – Indicates if this match was a bye (no opponent). In our data, `isBye: false` for all listed matches (since both teams played) [38].

Teams Involved:
- `match.home` – The Home Team object. Contains the same fields as a Team in standings (id, name, number, etc.) and additionally includes the **roster** (array of players) as discussed above [39] [40]. For example, home team might be `"Chalk and Awe"` with its roster listed.
- `match.away` – The Away Team object. Similarly structured, including roster. In the JSON, the `away` team is usually listed first, then `home`. For instance, in match 48118020, *Cue-ligans* was the away team and *Chalk and Awe* was home; the data for *Cue-ligans* (away) appears before the home team's data [41] [39]. Each team entry also has a nested `division` reference (id and type) and `league` reference for completeness [42] [43].

  - Both `home` and `away` team objects include their **team** `id` and other properties, which allows linking back to standings or team info. The inclusion of full roster under each team in the match is notable – it ensures the interface knows all players on both teams (useful for scorekeeping).

Match Logistics:
- `match.location` – The host location where the match was played. This is a **HostLocation** object including:
- `location.id` (Integer) – Location ID.
- `location.name` (String) – Venue name (e.g. `"DK's Stix & Stuff"`) [44] [45].
- `location.phone` (String) – Contact phone for the location [45].
- `location.address` – Address object with fields: `address1`, `address2`, `city`, `zip`, `latitude`, `longitude`, etc. [44]. In our example, address1 is *"1655 Ruben Torres Blvd #205"* in Brownsville, zip 78521 [44].
- These details can be used to display where the match took place or integrated with a map.

  - `match.fees` – An object detailing match fees for the week (for electronic payment records):
  - `fees.amount` (Number) – Base amount due (e.g. 45.00 USD for a team match) [46].
  - `fees.tax` (Number) – Tax amount if any (usually 0).
  - `fees.total` (Number) – Total amount (often equal to amount if no tax).
  - In our data, `"amount": 45, "tax": 0, "total": 45` indicates a $45 weekly team fee.
  - `match.orderItems` – If fees were paid online, this array contains an Order reference:
  - Each entry has an `order.id` and possibly the member who paid. In our example, there is one order item with an `Order.id: 1978562` paid by member Justin Ross [47] [48]. This shows Justin's firstName/lastName and member id in the order record. This ties into the payment system but isn't critical for standings; however, a web app might show "Paid by [Member]" for record-keeping.

Match Results and Scores:
The most detailed part of the match data is the breakdown of **results**. The `match.results` field is an array with two **MatchResult** entries – one for the home team and one for the away team. Each MatchResult includes:

  - `homeAway` (String) – `"HOME"` or `"AWAY"`, indicating which team this result corresponds to [49] [50].

- `matchesPlayed` (Integer) – Number of individual matches played in this team match (typically 5 in APA standard format) [49] [50]. If a match was forfeited or a team had a bye week, this might be less, but in our data it's 5 for both teams.
- `matchesWon` (Integer) – How many of those individual matches this team won [49] [50]. For example, home might have `matchesWon: 4` vs away `matchesWon: 1` if the home team won 4 out of 5 matches.
- `forfeits` (Integer) – Number of individual matches forfeited by the team [49]. (In our example, `forfeits: 0` for both sides, meaning all matches were played) [49].
- `overUnder` (Integer) – This likely relates to the **"Skill Level Over/Under"** differential. APA has a rule about total skill levels in a 5-person 23-rule for 8-ball. If a team couldn't comply or had to play shorthanded, this might be reflected. In our data it's `0` for both teams, indicating no over/under penalty applied [51] [52].
- `points` – A **MatchResultPoints** object summarizing points earned by this team in the match [53] [54]. It includes:
- `total` (Integer) – Total points earned in this match (including bonus points) [55] [56]. e.g. Home team total 14, Away team total 6 in one match.
- `won` (Integer) – Points from individual match wins (sometimes called "points won"). In APA 8-ball, each individual match win yields points based on the margin; here home `won: 11` vs away `won: 3` [55] [56]. The difference is often due to bonus points (explained next).
- `bonus` (Integer) – Bonus points awarded (often for submitting paperwork on time, etc.). In our example each team got `bonus: 3` [57] [56]. APA 8-ball typically gives 2 or 3 bonus points each week for proper scorekeeping; here it appears as 3.
- `penalty` (Integer) – Penalty points, if any deducted (0 in these cases) [57].
- `adjustment` (Integer) – Any manual point adjustments (0 here) [57].
- `sportsmanship` (Integer) – Possibly points related to sportsmanship ratings (0 here, not commonly used in standard scoring) [58].
- `skillLevelViolationAdjustment` (Integer) – Penalty points if a team violated skill level rules (0 here) [58].

Together, these fields explain the points: e.g. Home won 11 points from matches + 3 bonus = 14 total [58], Away won 3 + 3 bonus = 6 total [54]. These totals correspond to the "pointsLastWeek" that would appear in standings the next week for those teams.

- `scores` – This is an array of **MatchScore** objects detailing each individual matchup in the team match [59] [60]. Each MatchScore corresponds to one player's performance. Notably, each individual match between two players will produce *two* MatchScore entries: one for the player on the home team and one for the player on the away team. The key fields of a MatchScore are:
- `player` – Player object (id and displayName) for the person who played [61]. The `__typename` will match the format (EightBallPlayer/NineBallPlayer). This links to the roster players.
- `playerPosition` (Integer) – This seems to indicate which team's lineup slot the player filled, or a relative position in the pairing. In the data, one side's player might be position 1 and the other position 2 for the same match. (It's not simply home=1, away=2; it varies by who was put up first, etc. It might not be critical for display, aside from possibly ordering the scores).
- `matchPositionNumber` (Integer) – The match index (1 through 5) of this individual match in the overall meet [62] [63]. This allows pairing up the two scores representing the same table/match. For example, both the home and away score entries with `matchPositionNumber: 1` represent the first matchup of the night.

- `winLoss` (String) – `"W"` or `"L"`, indicating if this player won or lost their individual match [64]. Each pairing will have one W and one L.
- **Performance stats:** depending on format:
  - For 8-Ball:
  - `eightBallWins` – Number of games (racks) won by this player in that match [65]. APA 8-ball matches are won by reaching the opponent's handicap race number, but they track total games won; e.g. a player might win 4 games to 2. In our data, e.g. Steve Castillo (SL5) has `"eightBallWins": 4` [66], meaning he won 4 games in that match.
  - `eightBallMatchPointsEarned` – Points earned for the team from this individual match [66]. APA 8-ball scoring gives 3-0, 2-1, or 2-0 points depending on match outcome; in the example Steve earned 2 points for his team with that win [66]. His opponent Ethan (away side) got 1 point in that match (as seen in Ethan's score entry) [67] [68].
  - `eightOnBreak` – Whether the player made an 8-ball on the break (count of 1 if yes, else 0) [69]. All examples show 0 here.
  - `eightBallBreakAndRun` – Whether the player broke and ran (8-ball B&R, count) [70]. Also 0 in our data for each.
  - For 9-Ball (not explicitly shown in the snippet above, but analogous fields exist: `nineBallPoints`, `nineBallMatchPointsEarned`, `nineOnSnap`, `nineBallBreakAndRun`, etc., which would appear for NineBallPlayer entries). In our file, these are mostly null for 8-ball matches [63] [71].
  - `defensiveShots` – Number of defensive shots (safeties) the player executed (APA encourages recording of defensive shots) [72] [61]. Ex: Steve had 9 defensive shots [72], whereas some players had 0 or few.
  - `innings` – Number of innings (turns at the table) in that individual match [73] [74].
  - `doublesMatch` – Boolean if this was a doubles match (in our standard 5-person league, this is false for all) [75].
  - `mastersEightBallWins` / `mastersNineBallWins` – Fields relevant only in Masters format (not applicable here, hence null) [76].
  - `matchForfeited` – Boolean if this individual match was forfeited [62] (false for played matches).
  - `incompleteMatch` – Boolean if the match didn't finish (rare, e.g. ran out of time) [77]. All false in our data.

Putting it together, a MatchScore example for one pairing:

- Home player Steve Castillo (SL5) vs Away player Ethan Boone (SL6) in match #1:
- Steve (home): 4 games won, earned 2 points, winLoss "W" [66] [62].
- Ethan (away): 4 games won (he reached the hill but lost), earned 1 point, winLoss "L" [67] [68].
- Both had 46 innings, which matches (they played a drawn-out match), and Steve's defensive shots (9) + Ethan's (6) recorded safeties.

The JSON's structure lists all home team players' scores then away team players' scores separately under each team's MatchResult. However, the `matchPositionNumber` and other stats make it possible to pair them and reconstruct each individual match outcome. In a database, one might instead store an **IndividualMatch** record with references to both player IDs, their skill levels, and the stats above, rather than storing two separate records – but since APA's API splits them by team perspective, our JSON reflects that split.

Example excerpt with points and scores:

[53] [59]

This shows the home team got 14 points (including 3 bonus) and lists their five MatchScore entries. The away team (not shown in this snippet) has the complementary data (6 points total, their five MatchScore entries).

From an integration standpoint, the **Match ID** links to a division and week. Team IDs inside match link back to the Team entries in standings. Player IDs in scores link to the roster entries (which share the same player id and names). So a Flask app could use match data to display a detailed score sheet: listing each matchup with player names, scores, innings, etc. The presence of roster info means the front-end didn't need an extra call to get player names for those player IDs – it's already included.

Player Profile (Alias) and Lifetime Stats

When clicking on a player in the web app (or viewing one's own stats), the app fetches **member/alias details**. APA uses the concept of an **Alias**, which represents a member's identity within a specific league. Each alias has its own stats and history in that league. In our JSON, for example, the user clicked on a player which led to a page URL like `/member/1651980/851051/eight/137` – where `1651980` is the member ID and `851051` is the alias ID in that league, and `eight/137` indicates 8-ball format, session 137 (Fall 2025). The data returned for an alias includes:

- `alias.id` (Integer) – Alias ID (unique within the league). For instance, Gilbert Duran's alias id is `851051` [78]. (Often league code + member ID, but treated as an opaque ID here).
- `alias.displayName` (String) – The player's name as used in league (e.g. `"Gilbert Duran"`) [79].
- `alias.member` – Nested Member info if needed, such as `consecutiveYearsPlayed` and `membershipHistory`. In the `getMembershipHistory` query, for example, we see:
- `alias.member.id` (Integer) – global member ID.
- `alias.member.consecutiveYearsPlayed` (Integer) – e.g. number of years in a row the member has played.
- `alias.member.membershipHistory` – an array of past memberships. Each entry has a `year` and the `leaguePaidIn` (which league they paid their APA membership fee in that year) [80] [81]. This can show how long the player has been active and in which areas. (This data was partially visible in the JSON, showing structure but not fully expanded in our snippet).
- `alias.players` – This is an array of **Player objects** corresponding to each session and format the player has participated in (filtered by the query parameters `current` or `active`; in our case it seems to return recent sessions for that alias). Each entry in this array includes:
- `players[].id` – A player instance ID (like those seen in roster and match data) for that alias.
- `players[].session.id` – Session ID the player record is for [82]. For example, `session: {id: 137}` would be Fall 2025, and another entry might have `session: {id: 136}` for the previous session [83].

- Format-specific stats: If the entry is an EightBallPlayer or NineBallPlayer, it includes cumulative stats for that session:

    - `eightOnBreaks`, `eightBallBreakAndRuns`, `rackless`, `miniSlams` (for 8-ball) [84] [85].
    - `nineOnSnaps`, `nineBallBreakAndRuns`, `miniSlams`, `skunks` (for 9-ball) [86] [87]. These are totals for that session: e.g. Gilbert's data shows `nineOnSnaps: 1` in one entry (meaning he had one 9-on-the-snap that session) [82], and various 0s for others. These fields correspond to special accomplishments: 8-on-Break, Break-and-Run, Rackless nights, "Mini-slam" (both an 8OB and 8BR in one night), 9-on-Snap, 9BR, Skunk (shutout in 9-ball).
    - The `players` array essentially gives a quick summary of the player's recent performance in each session for each format. In our example for alias 851051, we see entries for Fall 2025 (session 137) for both 8-ball and 9-ball (since Gilbert plays both formats, perhaps as part of Double Jeopardy) and possibly the previous session 136 as well [88] [89]. Each has its own id and stats.

- **Lifetime Stats:** The query `alias.stats(filter: EIGHT)` and `alias.stats(filter: NINE)` return lifetime summary stats for that alias in the given format:

- For 8-ball (`EightBallLifetimeStatistics`): fields like `matchesPlayed`, `matchesWon`, `CLA`, `defensiveShotAvg`, `matchCountForLastTwoYrs`, `lastPlayed` [90] [91].
    - **CLA** (Numeric) – Stands for *Current Lifetime Average* (or possibly *Career Lifetime Average*, sometimes an internal skill metric). In the example, Gilbert has `CLA: 7` for both formats [92] [93], likely meaning he's a high skill level player.
    - **defensiveShotAvg** – Average defensive shots per match (1.86 for 8-ball, 2.29 for 9-ball in Gilbert's case) [92] [94].
    - **matchesPlayed / matchesWon** – Total lifetime matches and wins (e.g. 875 played, 479 won in 8-ball for this player) [95].
    - **matchCountForLastTwoYrs** – Number of matches in the last two years (38 in 8-ball) [96].
    - **lastPlayed** – Date of the last match played (ISO date) [97]. In the snippet, `lastPlayed: "2025-10-30T00:00:00Z"`.
- For 9-ball (`NineBallLifetimeStatistics`): similar fields (matches, wins, etc.) [98]. In Gilbert's case: 407 matches, 210 won in 9-ball [99].

These lifetime stats help gauge a player's experience and performance over their entire time in the league. The **alias ID** is the key to fetch these – for example, in Flask one might have an endpoint like `/players/<alias_id>` which pulls from an Alias table and related stats. The alias is also how team rosters and historical data connect: e.g. each roster entry is implicitly tied to an alias (though the JSON gives member and player session IDs, the alias ID can be derived or was known from the context of clicking the profile).

Normalization considerations: In a database, one would likely have tables for **Member** (global info), **Alias** (league-specific info), and then separate records for each **SessionParticipation** or **PlayerStats**. The alias object essentially is the parent for session-specific player records.

For example, consider Gilbert Duran:
- Member (id=1651980, name Gilbert Duran).
- Alias (id=851051, league_id=894, member_id=1651980).

- PlayerStats8 (id=91016696, alias_id=851051, session_id=137, format=8, matchesPlayed=4, matchesWon=2, SL=7, etc.) [100].
- PlayerStats9 (id=91016301, alias_id=851051, session_id=137, format=9, stats...) [88].
- LifetimeStats8 (alias_id=851051, totalMatches=875, wins=479, ...).
- LifetimeStats9 (alias_id=851051, totalMatches=407, wins=210, ...).

The JSON directly provides the data to populate these. For Flask endpoints, one might have:
- `/division/<id>/standings` to return teams and standings stats (using divisionStandings data).
- `/team/<id>` for team detail and roster (we might reconstruct from match or need a separate query – APA's API might not have a direct team detail query except via a match or custom query).
- `/match/<id>` for match scores (the MatchPage data).
- `/player/<alias_id>` for player profile (alias stats).

Because the JSON is effectively a dump of what the official site requested, it's already fairly **normalized by concept** (though sometimes duplicate info is present, e.g. roster appears in multiple match queries). A developer building their own app could choose to store this in a relational format for efficient queries or just fetch on demand via the APA API.

Summary of Key Entities and Fields (Reference Tables)

For clarity, here is a summary of the main entities derived from the JSON and their fields, with descriptions:

League (e.g. Rio Grande Valley APA):
| Field | Type | Description | |---|---|---| | **id** | Int | League ID (e.g. 894). Used to query divisions, etc. [2] | | **name** | String | League name (e.g. "Rio Grande Valley APA"). [3] | | **slug** | String | URL slug for league ("RioGrandeValley"). [2] | | **isElectronicPaymentsEnabled** | Bool | Whether ePayments are enabled for fees. [2] |

Division:
| Field | Type | Description | |---|---|---| | **id** | Int | Division ID (e.g. 417524). [4] | | **name** | String | Division name (often includes night/location, e.g. "Thu Brownsville 9 B"). [5] | | **number** | String | Division number/code ("222", "021", etc). [101] | | **format** | String | Format of play ("8-Ball Open" or "9-Ball Open"). [8] | | **nightOfPlay** | String | Night of week games are played (e.g. "Thursday"). | | **timeOfPlay** | String | Typical start time (e.g. "7:00 PM"). | | **isMine** | Bool | True if the logged-in member's team is in this division. [7] | | **state** | String | Division state (e.g. ACTIVE, FINISHED). | | **contacts** | Array | List of contacts (phone & alias for reps) [11]. Often empty. | | **league** | Object | Reference to parent league (id, slug) [8]. | | **schedule.id** | Int | ID of schedule (if needed to fetch full schedule separately). |

Team (as in standings or match context):
| Field | Type | Description | |---|---|---| | **id** | Int | Team ID (e.g. 12864635). [12] | | **name** | String | Team name. [102] | | **number** | String | Team number (division number + team code, e.g. "22202"). [14] | | **standing** | Int | Current rank in division (1 = first place). [15] | | **pointsLastWeek** | Int | Points earned in the most recent week. [13] | | **sessionTotalPoints** | Int | Cumulative points in the session. [18] | | **totalTeamMatchesPlayed** | Int | Team matches played so far (weeks played). [19] [20] | | **lastWeek** | Int | Last week number played (e.g. 11). [17] | | **isTied** | Bool |

Whether the team is tied in standings with another. [21] | | **isBye** | Bool | True if team is a dummy "Bye". [22] | | **league** | Object | Reference to league (id, slug). [21] | | **division** (if in team object) | Object | Reference to division (id, type) – seen in match context. [42] | | **roster** | Array | List of players on team (Player objects as below). |

Player (on roster):
| Field | Type | Description | |---|---|---| | **id** | Int | Player's session-specific ID (e.g. 91040482). [25] | | **displayName** | String | Player name. [25] | | **matchesPlayed** | Int | Matches played this session. [26] | | **matchesWon** | Int | Matches won this session. [26] | | **skillLevel** | Int | Current skill level (SL). [29] | | **pa** | Number | Points Average (APA stat). [27] | | **ppm** | Number | Points Per Match. [27] | | **member.id** | Int | APA Member ID (global person id). [28] | | **memberNumber** | String | APA membership number (often string like "785xxxxx"). [27] | | **__typename** | String | "EightBallPlayer" or "NineBallPlayer" (format context). |

Match:
| Field | Type | Description | |---|---|---| | **id** | Int | Match ID (unique for each team match). [34] | | **week** | Int | Week number of the session for this match. [35] | | **startTime** | String | Scheduled start datetime (ISO format). [35] | | **timeZone** | Object | Time zone info (id and name, e.g. CST). [36] | | **type** | String | Format of match ("EIGHT" or "NINE"). [36] | | **isFinalized** | Bool | True if scores are final/confirmed. [37] | | **isScored** | Bool | True if match has been scored (entries made). [38] | | **isPaid** | Bool | True if weekly fees paid online for this match. [38] | | **isTournament** | Bool | True if match is part of a tournament (league matches = false). [38] | | **isBye** | Bool | True if this match was a bye (one team had no opponent). [38] | | **home** | Team | Home team object (with id, name, number, roster...). [39] | | **away** | Team | Away team object (with id, name, number, roster...). [41] | | **location** | Object | Host location (id, name, phone, address). [44] [45] | | **fees** | Object | Match fees (amount, tax, total). | | **orderItems** | Array | If paid, order record(s) for payment (with order id and payer). [47] [48] | | **results** | Array | Two MatchResult objects (home and away results). | | **scoresheet** | (possibly Object/ID) | Was null in our data – could link to a PDF or image of scoresheet if available. [103] |

MatchResult (within results[]):
| Field | Type | Description | |---|---|---| | **homeAway** | String | "HOME" or "AWAY". [49] [50] | | **matchesPlayed** | Int | Number of individual matches played (usually 5). [49] | | **matchesWon** | Int | Number of individual matches this team won. [51] | | **forfeits** | Int | Number of matches forfeited by this team (counted as losses). [49] | | **overUnder** | Int | Skill level over/under points (if any handicap rule penalty). [51] | | **points.total** | Int | Total team points earned in this match (including bonuses). [57] | | **points.won** | Int | Points from individual match wins (before bonus). [55] | | **points.bonus** | Int | Bonus points awarded. [57] | | **points.penalty** | Int | Penalty points deducted. [57] | | **points.adjustment** | Int | Manual adjustment to points. [57] | | **points.sportsmanship** | Int | Sportsmanship points (if used). [104] | | **points.skillLevelViolationAdjustment** | Int | Points deducted for violating skill cap rules. [104] | | **scores** | Array | List of MatchScore objects for each individual match (for players on this team). |

MatchScore (player's performance in one individual match):
| Field | Type | Description | |---|---|---| | **player.id** | Int | Player ID (same as in roster) of the player. [61] | | **player.displayName** | String | Player name. [61] | | **playerPosition** | Int | Position indicator (1 or 2, relative in matchup). | | **matchPositionNumber** | Int | Match index (1–5) for pairing. [62] | | **winLoss** | String | "W" if this player

won, "L" if lost. [64] | | **eightBallWins** | Int | Games (racks) won by this player (8-ball). [65] | | **eightBallMatchPointsEarned**| Int | Team points earned from this match (8-ball). [66] | | **eightOnBreak** | Int | 8-on-Break count (usually 0 or 1) [69] . | | **eightBallBreakAndRun** | Int | 8-ball Break and Run count. [70] | | **nineBallPoints** | Int | Balls points scored (9-ball) – out of 20 (null for 8-ball). | | **nineBallMatchPointsEarned** | Int | Team points from this match (9-ball, 10-point system). | | **nineOnSnap** | Int | 9-on-the-Snap count (if 9-ball player). | | **nineBallBreakAndRun** | Int | 9-ball Break and Run count. | | **defensiveShots** | Int | Defensive shots (safeties) by this player. [72] | | **innings** | Int | Innings (turns) in this individual match. [73] | | **doublesMatch** | Bool | True if this was a doubles match (false in standard play). [75] | | **mastersEightBallWins** | Int | If Masters format, 8-ball wins (null in our context). | | **mastersNineBallWins** | Int | If Masters format, 9-ball wins (null). | | **matchForfeited** | Bool | True if this individual match was forfeited. [76] | | **incompleteMatch** | Bool | True if match started but not completed. | | **dateTimeStamp** | String | Timestamp when the score was submitted (or when match was played – our data shows same timestamp for all entries, likely submission time) [72] . |

(Fields that don't apply to the format are often `null` as seen in the JSON.)

Alias/Player Profile:
| Field | Type | Description | |---|---|---| | **alias.id** | Int | Alias ID (member's ID within league). [78] | | **alias.displayName** | String | Player's name. [79] | | **alias.member.id** | Int | Global member ID. | | **alias.member.consecutiveYearsPlayed** | Int | Number of consecutive years the member has played. | | **alias.member.membershipHistory[]** | Array | Years of membership and league info [81] . | | **alias.players[]** | Array | Session-specific player records for this alias. | | *alias.players[i].id* | Int | Player ID for that session (as seen in rosters/matches). [88] | | *alias.players[i].session.id*| Int | Session ID for this record. [105] | | *alias.players[i].__typename*| String | "EightBallPlayer" or "NineBallPlayer". | | *EightBallPlayer fields* | - | e.g. `eightOnBreaks`, `eightBallBreakAndRuns`, `rackless`, `miniSlams` (totals for that session) [85] . | | *NineBallPlayer fields* | - | e.g. `nineOnSnaps`, `nineBallBreakAndRuns`, `miniSlams`, `skunks` [86] . | | **alias.EightBallStats** | Array | Lifetime 8-ball stats (usually a single-item array) [78] . | | **alias.NineBallStats** | Array | Lifetime 9-ball stats (single-item array) [106] . | | *EightBallLifetimeStatistics:* | - | `matchesPlayed`, `matchesWon`, `defensiveShotAvg`, `CLA` (current lifetime average/skill), `matchCountForLastTwoYrs`, `lastPlayed` [91] . | | *NineBallLifetimeStatistics:* | - | similar fields for 9-ball [93] . |

These detailed profile stats are used for player rankings, milestone tracking (e.g. 500 matches played), and showing improvement over time. In a web app, you might display these on a player's profile page.

Integration Notes for a Flask App

Data Modeling: Given the relationships observed, a normalized design would have tables for core entities like League, Division, Team, Member, Alias, Session, Match, PlayerStats, and MatchPerformance. For example:

- **League** – stores leagues (id, name, slug).
- **Division** – stores division info (id, league_id, name, format, etc., plus maybe session_id if divisions reset each session). APA division numbers often reset each session, so including `session_id` in Division might be wise if the ID alone isn't globally unique (though in our data the division IDs are unique and likely encode session).

- **Team** – stores teams (id, division_id, name, number). Team number plus division can identify a team as well.
- **Alias** – stores alias (id, member_id, league_id, displayName).
- **PlayerStats** – could be broken by format, or a single table with fields for both 8-ball and 9-ball stats as applicable. It might include alias_id, session_id, format, matchesPlayed, matchesWon, skillLevel, and cumulative stats (break and runs, etc.). The roster can be drawn from this table (filter by team & session). However, team membership might be needed: perhaps a join table **TeamMember** with alias_id, team_id, session_id, plus their stats fields. Since the roster data ties a player to a team in a session, that's likely appropriate.
- **Match** – stores match records (id, home_team_id, away_team_id, session_id, week, startTime, location_id, isFinalized, etc.).
- **MatchResult** – possibly a table linking team and match, with points and matchesWon (so two rows per match, or one row with both teams? But easier as two rows or a JSON field). Alternatively, store home_points, away_points in Match, but APA's system of bonuses and such might be better in a structured way.
- **MatchPerformance** – individual match performances. One design: a table for individual matchups with fields: match_id, matchPosition, home_player_alias_id, away_player_alias_id, home_score (points), away_score, innings, defensiveShots_home, defensiveShots_away, etc. However, given APA's data is already split by player, one might just store each player's performance separately and infer pairings. A cleaner approach is to pair them in one record.

The data we have provides everything to populate such a database. For example, after parsing:
- Insert league 894.
- Insert divisions (417524, 417530, etc.) with their properties.
- Insert teams for those divisions with standings info. Update their points after each week or store weekly points history if needed.
- Insert alias and members for every player encountered in rosters (ensuring no duplicates).
- Insert team-member relationships for rosters.
- Insert matches and then insert match performances.

Endpoints Planning:
- **League list/selection** – not as necessary for a single-league app, but if multi-league, an endpoint could list leagues from viewer.
- **Divisions endpoint** – e.g. `/leagues/<league_id>/divisions` to list divisions (id, name, format, etc.). This can be populated from the divisions query [107] [108].
- **Standings endpoint** – e.g. `/divisions/<id>/standings` returning teams with fields like name, rank, points, etc. This comes directly from the `division.teams` data [16] [109].
- **Team detail endpoint** – e.g. `/teams/<id>` could return team info plus roster (players and their stats). Since the APA API doesn't give a direct team query, we gather roster from either a match or by querying the alias of each player. But our data from match pages already gave us rosters. We might use the latest match or a separate data source for roster. In our `api_dump`, the roster appears via match queries. Alternatively, one could reconstruct roster by taking all players from all matches of that team (the union of players who played), but that might miss bench players who haven't played yet. The APA official app likely has a roster component we saw (the fragment `rosterComponent on Team` in the GraphQL queries) [110] [111] which suggests there is a direct way to fetch roster by team ID. If building our own, we might call a similar query or use the data from the first match where the team played. In our case, since we scraped all matches for *Cue-ligans*, we effectively collected their full roster.
- **Match endpoint** – `/matches/<id>` to get detailed match info: teams, scores, etc. This directly maps to

the MatchPage data. We'd include everything from team names to individual scores. The JSON can be used almost as-is to present a scoresheet. We would likely process it into a more convenient shape (like an array of 5 matchups with both players' names, skill, scores).
- **Player endpoint** – `/players/<alias_id>` for player profile, returning their stats (lifetime and current session). We have the alias queries providing that [91] [106]. Possibly separate sub-endpoints for format if needed (or just return both 8-ball and 9-ball stats).

APA Concepts Mapping:
- *Division vs Session:* APA sessions (Spring, Summer, Fall) are separate seasons. Divisions are essentially tied to a session. The data implies division IDs might be unique per session (since currentSessionId is 137 for Fall 2025 in league info [112]). We see that `league.currentSessionId = 137` and all our division IDs correspond to session 137 divisions. Past session's divisions might have different IDs. So in the database, Division could include session_id or session as part of key.
- *Alias vs Member:* The separation of alias (851051) and member (1651980) is important. If the app might later integrate multiple leagues, a member can have multiple aliases. In our single-league scope, alias is basically the player's ID in this league. It's safe to use alias id as primary key for players in our league-specific app, but keep track of the underlying member ID in case of future expansion (or for profile details like membership history which is under member).
- *Points system:* The breakdown of points, match wins, etc., could be encapsulated in logic (to compute standings or match outcomes). But since APA has some unique scoring, it's useful that the API provides the computed values (we can trust `sessionTotalPoints` in standings and `points.total` in match results).

Conclusion: The `api_dump.json` is a comprehensive capture of APA league data, covering everything from league setup to individual match performances. By examining each part – **leagues -> divisions -> teams -> matches -> players** – we see a relational structure that a Flask app can mirror. We have identified the keys (IDs for league, division, team, alias, match) that connect these entities. With this analysis, one can design database models that correspond to these concepts and create API endpoints or pages that display standings, team rosters, match scorecards, and player stats. Each JSON field's purpose has been tied to an APA concept (e.g., skill levels, points, break and runs), ensuring that our Flask app can correctly interpret and present the information to users.

---

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 api_dump.json

file://file_000000008f1871f5be0dd19aca280caf